

Credit Name: CSE3010 - Object-Oriented Programming 2
Assignment Name: Account, PersonalAcct, BusinessAcct
How has your program changed from planning to coding to now? Please explain?
Understanding The Problem
I was to make a banking program that allowed the user to create a personal or business account with a fee applied if the account's balance was below \$100 for personal accounts, applying a \$2 fee, or below \$500 for business accounts, applying a \$10 fee. I was also supposed to make it able to change the address of the account. I had trouble trying to figure out how the fee was applied. I first thought of it as a monthly fee, however, it is not in my skill level right now to integrate time into my code. After clarifying with Mr. Abdalla, the fee was applied per withdrawal.
Planning The Solution/Implementation
I started with creating and altering any methods from the given code to include an address(street, city, province and postal code). I then created a class for personal accounts that inherits from the <i>Account</i> class to apply the fee. I created a method that will withdraw(method from <i>Account</i> class) the fee amount if the account's balance, retrieved using <i>getBalance()</i> (method from <i>Account</i> class), is less than the minimum amount of \$100. If the account's balance is less than the minimum amount, \$2 is withdrawn as the fee. I repeated these steps to create a class for business accounts; withdrawing \$10 if the account's balance is less than \$500. In the client code, I created a method that returns an object, prompting the user to enter all the information needed to make an account(first name, last name, initial balance and address) before asking whether it is a personal or business account. I used a while loop to continue prompting the user for a valid account type to ensure an account is created. I then returned an object based on what the user picked. In the main method, I displayed the possible actions: creating an account, withdrawing/depositing, viewing account details, changing account address and ending the program. If the user wanted to create an account, I used the object method created earlier. If an object if returned, it checks whether it was a personal or business account. If it was a personal account, I used a for-loop to check if another account with the same account ID already exists in the personal account arraylist. If it doesn't exist, the personal account object is added to the arraylist and a message explaining the minimum account balance requirement. I then repeated the steps for the business account. If the user wanted to withdraw/deposit, I prompted for the account ID and account type. It was separated by depending on the account type; if it was a business account, I created a method that checked to see if the account ID exists in the business account arraylist using a for-loop. If the account exists, it would proceed and ask the user if they wanted to withdraw or deposit into the account. For withdrawal, I prompted the user for a amount and used <i>withdrawal()</i> as well as <i>BusinessMinFee()</i> to apply or not apply the fee. If the account was not found, it would display so. I repeated this for the personal account. If the user wanted to view account details, it was split between personal and business accounts. If it was a business account, I would once again use the method to check if it exists in the arraylist before displaying the <i>toString()</i> . I repeated it for personal accounts. If the user wanted to change the account's address, I prompted for the account ID and account type, using the method once again to check if it exists based on whether it was a business or personal account. If it exists, I prompt the user for the new address and use <i>changeAddress()</i> from the <i>Account</i> class to replace the address.
Overcoming Challenges
The hardest part was getting the code to function properly due to the amount of nested if statements and while loops, especially before I made methods to check if the business and personal accounts exists. It took a lot of attention to detail to ensure that the brackets lined up properly and that the code was in the proper section to display correctly. It also took a bit to figure out how the logic should flow, especially before making the methods. With a little patience and testing to ensure that everything displayed properly, I was able to clean up the code.
Learning
With how frustrating testing code can be due to the number of possibilities, I am now able to efficiently test my code. Being able to create methods to help clean up my code is also a good skill to have.